

Improving Simplifiable SUSP Searching Algorithms In CUDA



RESEARCH PROBLEM

- A puzzle is an array that includes only 1s, 2s and 3s. Some special puzzles (SUSP) help perform matrix multiplication faster. The larger a SUSP is, the faster the matrix multiplication algorithm is.
- To search for SUSPs, computer has to simplify millions of puzzle.
- My summer research project aims to optimize the simplify operation by taking advantages of using GPU, so that the algorithm can simplify more puzzles in a fixed amount of time.
- With faster simplifying algorithm, we can search for a larger SUSP, which helps performing matrix multiplication faster.

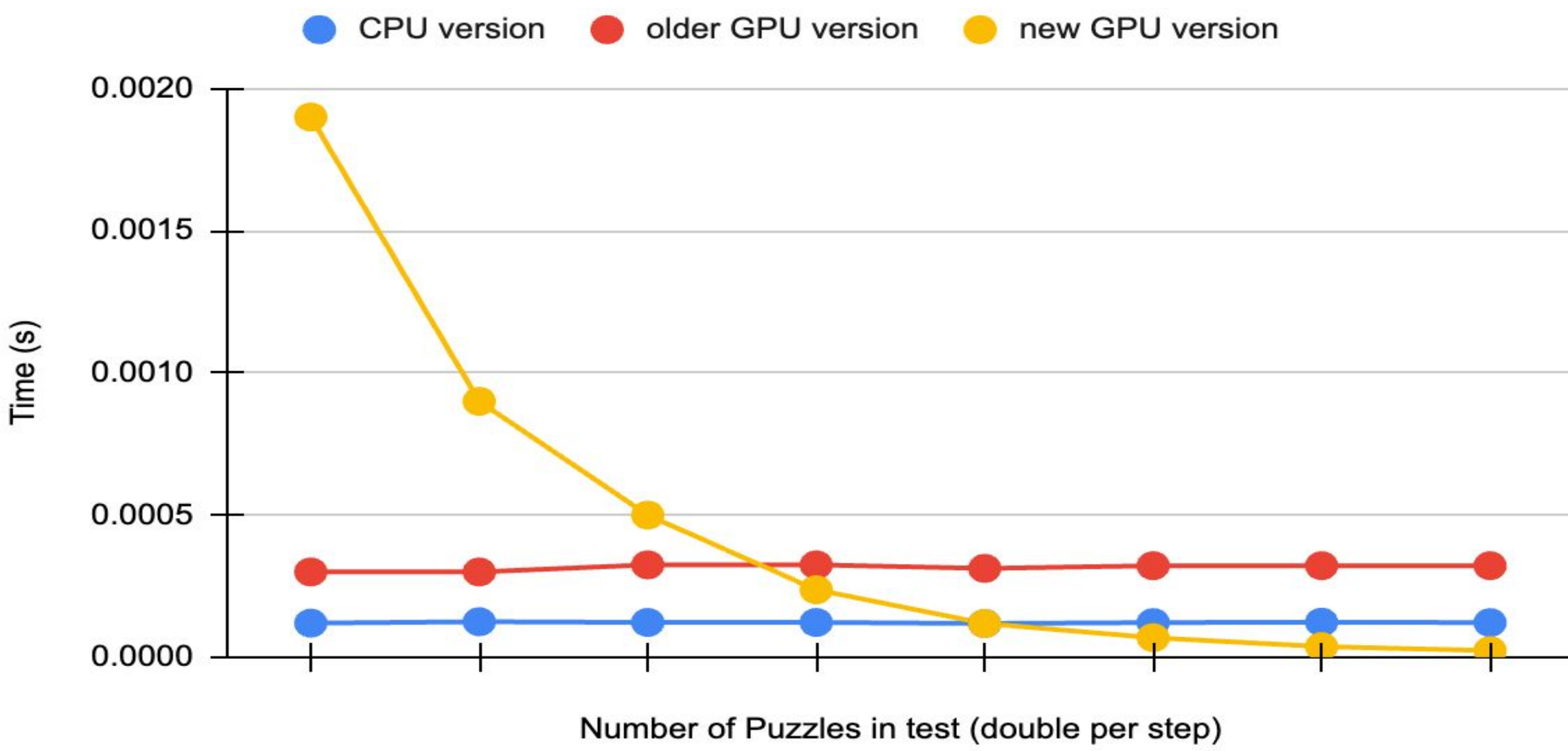
3	2	3	2
1	1	3	2
1	2	1	3
3	1	1	3
1	3	2	1

FIG 1: A puzzle

WORK ON CPU vs ON GPU

- CPU can simplify puzzles fast but can do only 1 puzzle per core at once. The execution time increases steadily with the number of puzzle.
- GPU, in contrast, has much more cores so it can simplify more puzzles at once.
- Therefore, the total execution time on GPU increases slower than CPU.
- As the project grows bigger, the field to search for SUSPs is growing too. We want to use all our available resources to boost the the algorithm's performance.

Average time to simplify 1 puzzle size 20 x 12



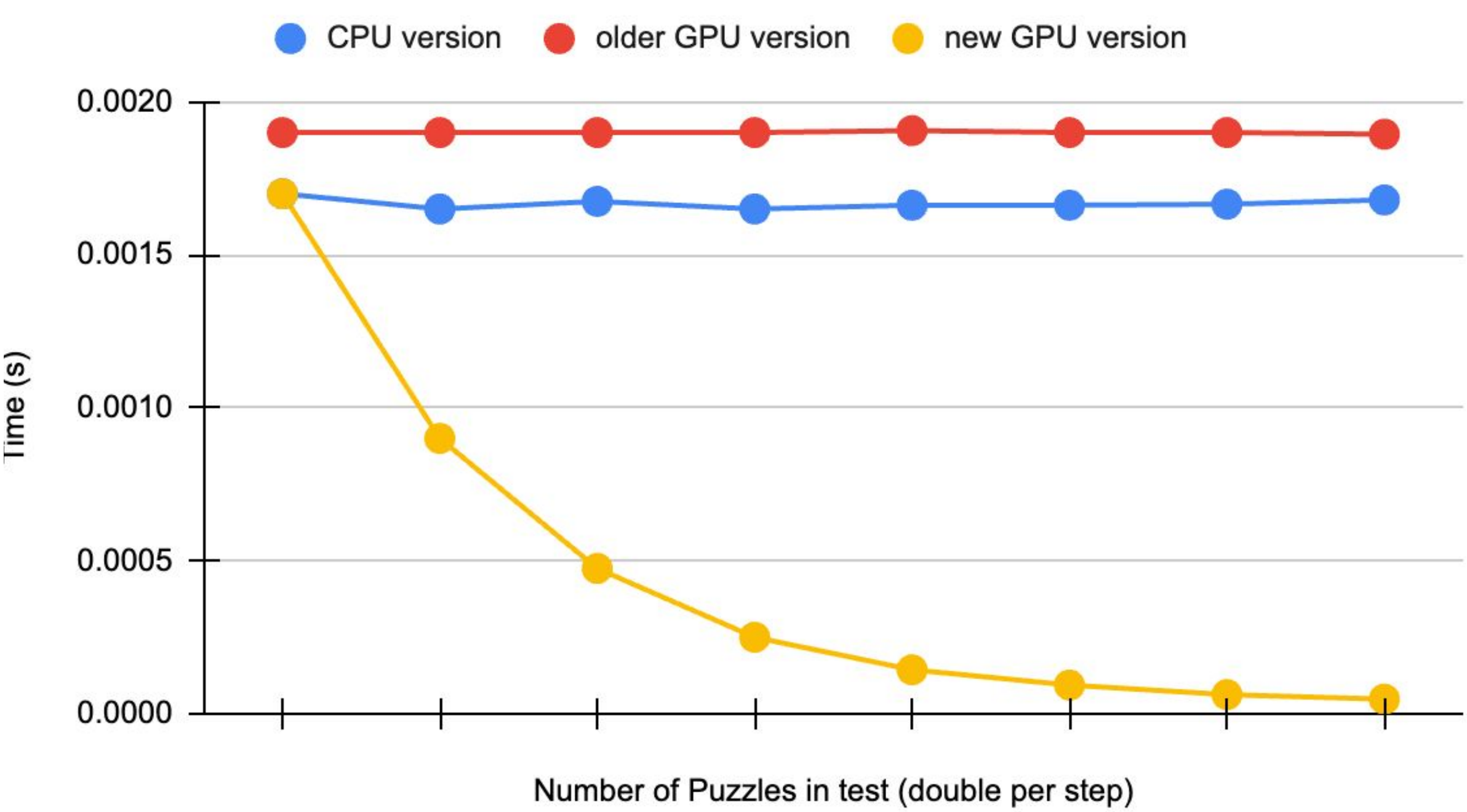
SIMPLIFICATION FOR PUZZLES

- Based on the old simplifying algorithm in [1], I implemented a CUDA program that simplifies a batch of 256 puzzles at once.
- First, each puzzle is converted into a Three-Dimensional Matrix (TDM). Each entry represents whether a triple of rows is still possible.
- The GPU then repeatedly checks the TDM from three different projection faces.
- For each face, the code builds a 2D projection, finds which row pairs can still stay together, and removes impossible entries from the 3D matrix.
- This process repeats until the puzzle no longer changes for all three faces, or until the maximum number of iterations is reached.
- Finally, the code counts the remaining valid entries in the TDM. A smaller remaining TDM means the puzzle has been simplified more.

OPTIMIZATIONS IMPLEMENTED

- Persistent GPU resources: Device buffers are allocated once through class GpuResource, reducing repeated cudaMalloc / cudaFree overhead.
- Faster memory transfer: Using pinned host memory, cudaMemcpyAsync, cudaMemcpySetAsync, and CUDA streams to reduce CPU-GPU waiting time.
- Optimized TDM computation: The check is rewritten with bit manipulation for $k < 16$, replacing slower character-by-character check in the baseline.
- Shared memory projection: The graph simplification kernel loads each 2D projection into shared memory, reducing repeated global memory access.
- Less CPU synchronization: Stopping conditions are handled on GPU, avoiding CPU copy-back after every iteration.
- CUB block reduction: Final TDM count uses cub::BlockReduce and atomic accumulation, replacing older multi-kernel sum plus final CPU summation.

Average time to simplify 1 puzzle size 50 x 15



INTEGRATING INTO SEARCH PROGRAM

- GPU-Accelerated Candidate Evaluation: The enhanced version shifts the heavy calculations away from the CPU, offloading these tasks to the GPU. This integration maximizes GPU utilization, which is critical for efficiently searching program.
- Batched Workload Processing: The enhanced search loop batches multiple neighborhood states. This approach saturates the CUDA cores and hides memory latency by minimizing expensive host-to-device transfers.
- Asynchronous Hybrid Pipeline: The CPU maintains control over the high-level Iterated Local Search stages, while issuing asynchronous computation tasks to the GPU, keeping both processors continuously fed.
- Multi-GPU Architecture: By utilizing a producer-consumer workflow, the main controller continuously queues evaluation tasks, while multiple GPUs asynchronously process these workloads, ensuring constant hardware saturation across all available GPUs.

OTHER IMPLEMENTATIONS

- Checkpointing Mechanism: The save point mechanism enables the program to pause and serialize its current state into a binary file. The program can resume the search progress later by passing the saved checkpoint file as a command-line argument.
- Bloom Filter: The program manages previously evaluated states and prevents redundancy using an isomorph cache with bloom filter. If a Bloom filter is utilized to optimize this duplicate checking, the program will use less RAM memory to store checking table under hash table. It has the probability to create false positives, so researchers have to make a trade off between performance and correctness.

```
s: 30, #: 100 ( 94, r: 0), Gap: -1.2000e+01,  
Q: 104646 [-1.2755e+01 .. -2.9264e+07] (Prev Stages: 2 <- 0 <- 3)  
Distinct: 2972, Repeated: 25  
Eval: 4.9152e+05 (per sec: 4787),  
GPU Simplify: 172 ( 0%) (per sec: 1.68)  
CPU Simplify: 351634 (100%) (per sec: 3424.77)  
Tot Simplify: 351806 (100%) (per sec: 3426.44)
```

FIG 2: Metric of traditional searching process

```
s: 30, #: 170 ( 31, r: 0), Gap: -1.8000e+01,  
Q: 209586 [-1.8748e+01 .. -2.9264e+07] (Prev Stages: 0 <- 1 <- 5)  
Distinct: 3057, Repeated: 31  
Eval: 1.0130e+06 (per sec: 10081),  
GPU Simplify: 1504955 ( 97%) (per sec: 14976.82)  
CPU Simplify: 43217 ( 3%) (per sec: 430.08)  
Tot Simplify: 1548172 (100%) (per sec: 15406.90)
```

FIG 3: Metric of enhanced searching process with GPU

RESULT & CONCLUSION

- With the same amount of work, new enhanced version takes significantly short time to simplify 1 puzzle than two older version. The processing time of the new algorithm can be up to 40 times faster than the old algorithm in tests with big Puzzles.
- In the full search program, the enhanced version completes more total simplification work per second, with nearly 5 times speed up.
- With the enhanced searching program, we were able to find the new SUSP of size 79x10, surpassing the previous record (78x10).
- For the next step, we plan to try some new methods, like apply Machine Learning to reduce amount of puzzle required to simplify.

References

[1]: Anderson, Matthew & Le, Vu. (2023). Efficiently-Verifiable Strong Uniquely Solvable Puzzles and Matrix Multiplication. 10.1007/978-3-031-49193-1_4.